

Baseball/Softball Lineup Optimization System:

A Five-Module AI Pipeline for Lineup Construction and Adaptive Game Planning

Kali Javetski, Sylvia Burroughs

Demo: http://127.0.0.1:8765/module4_dashboard.html

Repository: <https://github.com/Alvin-Furman-CS-Classroom/project-2-ai-system-bases-loaded-with-knowledge>

Abstract

Lineup management in baseball and softball requires balancing offensive matchups, defensive reliability, positional constraints, batting order, and real-time game conditions — a problem that resists both pure intuition and simple optimization. This paper presents a five-module AI pipeline designed to support coaches with pre-game lineup construction and in-game tactical adaptation. The system combines first-order logic for batter-pitcher matchup scoring, a knowledge-based approach for position-specific defensive evaluation, constraint satisfaction for legal positional assignment, a genetic algorithm for batting order search, and a planning module that converts lineup state into prioritized multi-inning recommendations.

Each module exposes clear input-output interfaces, enabling independent testing and reliable pipeline chaining. Evaluation focused on implementation correctness, cross-module handoff integrity, and demonstration readiness rather than external predictive benchmarking. The test suite totals approximately 246 unit-test methods across five modules and three integration chains covering the full Modules 1–5 execution path. All integration tests passed, and the system was successfully demonstrated through a browser dashboard supporting live replanning. Results show that combining symbolic reasoning, constrained search, evolutionary optimization, and forward planning produce a coherent, inspectable decision-support tool. The system does not automate coaching judgment, but provides structured, stage-by-stage recommendations with clear tradeoffs at each step.

1. Introduction

Constructing a lineup in baseball or softball is not a single decision — it is a sequence of interdependent choices. A coach must determine who is eligible at each position, how to balance offensive production against defensive reliability, and how to order nine players across a batting lineup that interacts with opponent pitching. During a game, those choices evolve: a one-run lead in the seventh inning calls for different priorities than a three-run deficit in the third.

This project formalizes that decision process as a five-stage AI pipeline. Each stage addresses a distinct subproblem — offensive scoring, defensive scoring, positional assignment, batting order optimization, and game-state-aware planning — with structured interfaces connecting them. The goal is a system that is testable at each stage, interpretable in its outputs, and coherent as an integrated whole.

The scope is deliberately bounded. The system works from player statistics and game context provided by the user; it does not scrape live data or interface with external APIs. The planning module accepts a pitcher fatigue field for forward compatibility but does not issue pitching-change or bullpen recommendations, as no dedicated pitching module was built. These boundaries are design decisions, not oversights, and they are reflected throughout the evaluation.

2. System Architecture

The pipeline moves linearly through five stages, with each module's output serving as structured input to the next. Figure 1 illustrates the full data flow.

Modules 1 and 2 operate independently. Module 1 evaluates each batter's expected offensive output against the opposing pitcher. Module 2 scores each player's defensive value at each eligible position. Neither depends on the other, which simplifies testing and allows to be updated without affecting the other's logic.

Module 3 takes both score maps and a positional eligibility table, then solves a constraint satisfaction problem to assign nine players to nine positions. Module 4 receives that assignment and searches the space of batting order permutations — 9! candidates — using a genetic algorithm. Module 5 closes the loop: given the finalized lineup, current game state, and bench availability, it produces a prioritized set of tactical recommendations and a multi-inning plan.

Rather than sharing mutable global state, each module receives explicit inputs and returns explicit outputs. This made integration testing straightforward — the output of Module 3, for instance, could be validated against Module 4's input schema before any end-to-end run was attempted. Three integration chains cover Modules 1–3, Modules 3–4, and the full Modules 1–5 path.

Figure 1. End-to-End AI System Architecture

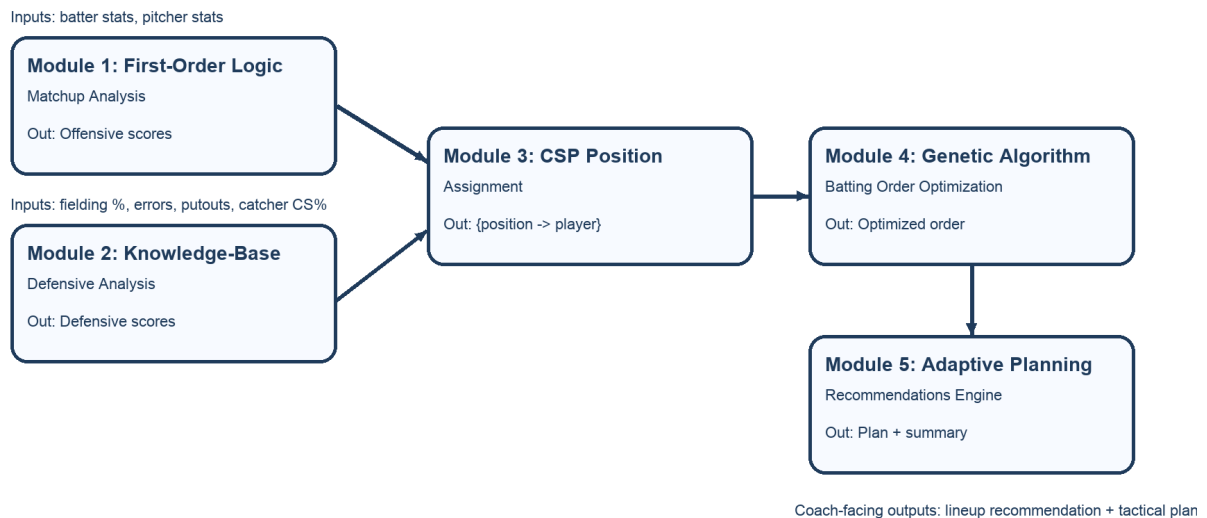


Figure 1. End-to-end system architecture and data flow. Each module transforms structured inputs into outputs consumed by the next stage.

3. Module Implementation Summary

3.1 Module 1 — First-Order Logic Matchup Analysis

Module 1 scores each batter's expected offensive performance against a specific opposing pitcher. Inputs are batter statistics (BA, OBP, SLG, HR, RBI, handedness) and pitcher statistics (ERA, WHIP, strikeout rate, walk rate, handedness). Output is a normalized score map over all available batters [1], [2].

The scoring logic uses first-order rules that reflect established baseball reasoning: a batter with high OBP against a pitcher with a high walk rate earns a bonus; platoon advantages based on handedness are applied as multipliers. Because the rules operate on statistical profiles rather than head-to-head history, the module generalizes to matchups with no prior data.

3.2 Module 2 — Knowledge-Base Defensive Analysis

Module 2 scores each player's defensive value at each position they are eligible to play. Inputs are fielding percentage, error counts, putouts, and, for catchers, additional metrics including passed balls and caught-stealing percentage [3], [4]. Output is a position-keyed score map per player.

The position-specific logic reflects the different demands of each role. Catcher evaluation weights arm strength and game-management metrics; corner infield and outfield positions emphasize fielding percentage and range proxies. This prevents a strong-fielding catcher from scoring equivalently to a strong-fielding left fielder, which a position-agnostic model would incorrectly produce.

3.3 Module 3 — CSP Defensive Position Assignment

Module 3 frames lineup selection as a constraint satisfaction problem with an embedded objective function. Inputs are the offensive score map from Module 1, the defensive score map from Module 2, and each player's positional eligibility list. Output is a single valid assignment of players to positions.

Hard constraints enforce two rules: one player per position and each assigned player must be eligible at their assigned spot. The objective maximizes a weighted sum of offensive and defensive scores, with configurable weights. An optional defensive leverage profile — drawn from FanGraphs positional adjustment data [5], [6] and sabermetric analysis of the defensive spectrum [11] — scales defensive scores by positional difficulty. Using CSP here guarantees a legally valid lineup while still optimizing for quality, which a greedy assignment would not reliably achieve.

3.4 Module 4 — Genetic Algorithm Batting Order Optimization

Module 4 optimizes the batting order for the nine players assigned by Module 3. The search space is $9! \approx 362,880$ orderings; a genetic algorithm was chosen because it scales naturally and produces interpretable fitness histories. Inputs are the assigned players, their batting statistics, and optional hyperparameters for population size, mutation rate, and stopping criteria. Output is the best-found batting order and associated fitness metadata.

The fitness function rewards placing high-OBP batters at the top of the order and concentrating power production in the middle slots, following established batting order theory [1], [7], [12]. Seeded runs and fitness-stagnation early stopping support reproducible results and controlled runtimes.

3.5 Module 5 — Adaptive Planning

Module 5 generates tactical recommendations for a configurable forward window, defaulting to several upcoming innings. Inputs are the current game state (score, inning, outs, base runners), the active lineup from Module 4, bench player roles and attributes, and the offensive and defensive score maps from Modules 1 and 2. Outputs are a prioritized recommendation list, a multi-inning tactical plan, and a game-state summary.

Candidate actions — positional substitutions, batting order adjustments, pinch-hit triggers — are scored on impact, urgency, and feasibility. For example, in a one-run lead entering the seventh inning, the planner prioritizes defensive substitutions at premium positions over offensive upgrades. Deterministic tie-breaking ensures identical inputs produce identical outputs. Bench-conflict resolution runs before final output, preventing the same player from appearing in multiple recommendations simultaneously.

4. Evaluation Methodology

4.1 What Was Evaluated

Evaluation addressed five questions: Does each module produce correct outputs for valid inputs? Do modules chain correctly so that one module's output is a valid input to the next? Are outputs well-structured and schema-consistent? Is behavior deterministic under repeated identical inputs? And does the system produce useful, non-conflicting recommendations in a live demonstration?

4.2 Metrics and Criteria

The project does not claim external predictive validity — no ground-truth game outcomes were used to score recommendation quality. Instead, correctness was assessed through test pass rates, coverage of rules and constraint cases, and rubric-aligned checkpoint reports [9], [10]. These reports document criteria-level findings, known weaknesses, and action items for each module.

4.3 Test Setup and Data

Tests are organized into unit tests/ and integration tests/ directories, with representative player and game scenario data in test data/. The approximate unit-test method distribution is Module 1 (~132), Module 2 (~67), Module 3 (~22), Module 4 (~13), and Module 5 (12), totaling approximately 246 methods across five modules [10]. Three integration chains provide end-to-end coverage. Table I summarizes coverage targets and test types per module.

The earlier modules carry more unit-test weight because they contain more independent logical rules to verify. Later modules, whose behavior depends heavily on upstream inputs, were validated more thoroughly through integration chains. This distribution was intentional rather than a gap in coverage.

Table I. Evaluation matrix showing unit test counts, integration chain coverage, and key correctness criteria per module.

Module	Unit Tests	Integration	Coverage Focus	Determinism	Key Criteria
Module 1 Offensive FOL	~132 methods	M1–3 chain	Rules, matchup logic, handedness, edge cases	Stateless scoring	Rule coverage, score range validity

Module	Unit Tests	Integration	Coverage Focus	Determinism	Key Criteria
Module 2 Defensive KB	~67 methods	M1–3 chain	Position heuristics, catcher metrics, fielding	Stateless scoring	Heuristic correctness, position differentiation
Module 3 CSP Assignment	~22 methods	M1–3, M3–4	Constraint satisfaction, eligibility, weighting	Deterministic CSP	Legal lineup guarantee, objective quality
Module 4 GA Batting Order	~13 methods	M3–4, M1–5	Permutation search, fitness, mutation, elites	Seeded GA runs	Fitness convergence, order validity
Module 5 Adaptive Planning	12 methods	M1–5 full	State validation, rule behavior, bench conflicts, output structure	Tie-breaking rules	Non-empty recommendations, conflict-free bench usage

4.4 Result Collection

Results were collected from three sources: automated test runs (unit and integration), checkpoint rubric reports documenting module-level quality findings, and live demo execution including dashboard rendering and replanning requests. This approach keeps reported claims tied to observable, reproducible evidence.

5. Results

5.1 Integration and Pipeline Outcomes

All five modules were completed with working interfaces, and all three integration chains passed. The Modules 1–3 chain confirmed that offensive and defensive score maps feed correctly into constraint-based assignment. The Modules 3–4 chain confirmed that a valid positional assignment is correctly consumed by the genetic algorithm. The full Modules 1–5 chain confirmed that the pipeline produces structured, non-empty planning outputs from raw player and game-state inputs [10].

No integration failures were present at final submission. The architecture's explicit interface contracts made it straightforward to isolate and fix handoff issues when they arose during development.

5.2 Testing Outcomes

The test suite spans approximately 246 unit-test methods and three integration chains. Module 1 has the highest coverage density (~132 methods), reflecting the number of independent logical rules in the matchup scoring system. Module 5 has the smallest unit suite (12 methods) but is covered by all three integration chains, which test end-to-end behavior more directly than isolated unit testing [10].

Module 5 checkpoint reporting documented passing tests for `test_game_state`, `test_strategy_rules`, and `test_planner`, and confirmed that the full Modules 1–5 integration run produced valid, explainable, conflict-free recommendation output [9].

5.3 Demonstration Outcomes

A browser dashboard was built to render lineup assignments, batting orders, and planning recommendations. A local planning server route supports context edits and replanning without restarting the pipeline. These tools served as behavioral evidence during demonstration: recommendation objects, inning-window summaries, and bench-conflict safety were verified in rendered output and API responses [9]. Figure 2 provides a representative view of Module 5 recommendations organized by inning context and game situation.

Table II. Representative recommendation types produced by Module 5, by inning window, score context, and priority level.

Inning	Recommendation Type	Score Context	Example Action	Priority Level
1–3	Offensive alignment	Tied / early deficit	Move high-OBP batter to lead-off slot	High (8–10)
4–6	Positional substitution	Lead or close game	Insert bench player at weak defensive position	Medium (5–7)
4–6	Batting order adjustment	Mid-game, fatigue signal	Promote power hitter to cleanup vs. opposing LHSP	Medium-High (6–8)
7–9	Defensive upgrade	Narrow late-game lead	Replace outfielder with defensive specialist to protect lead	High (8–10)
7–9	Pinch-hit trigger	Late, runners on base	Bench pinch-hitter matched against relief pitcher handedness	High (9–10)

5.4 Strengths and Weaknesses

The pipeline's primary strength is coherence: five distinct AI paradigms connect through well-defined interfaces, each contributing to a single decision-support workflow. Outputs at every stage can be inspected and reasoned about independently, which aids debugging and makes the system useful in practice.

The main weaknesses are scope driven. The planning module lacks pitcher and bullpen management, limiting its usefulness in late-game situations where pitching decisions dominate. Unit-test coverage is uneven across modules, with later modules relying more heavily on integration testing. Recommendation quality is also sensitive to input data completeness — missing or inconsistent statistics can degrade scoring and cascade through the pipeline.

6. Proposal Delta

Delta 1: Planning Scope Clarification.

The original proposal did not specify that pitching recommendations were out of scope. The final system makes this boundary explicit: the planning module accepts pitcher fatigue as an input field for forward compatibility but does not act on it, and no pitching recommendations are issued.

Delta 2: Enhanced Demo Integration.

The final system includes a browser dashboard and a local replanning server route not described in the proposal. These were added to improve demonstration quality and make outputs more inspectable during evaluation.

Delta 3: Evaluation Emphasis.

The proposal anticipated more unit-test depth in later modules. In practice, later modules were validated more heavily through integration chains, which provided stronger correctness evidence given their dependency on upstream outputs.

Delta 4: Interface Documentation.

Module specifications were expanded during development to tighten input-output contracts, prompted by integration issues discovered during testing. There were moments where we wanted to add in adaptations and possibilities for outcomes from our modules that were different than normal baseball outcomes. All five modules remained in the final system; none were dropped or merged.

7. Limitations and Failure Analysis

Limitation 1: No Pitching or Bullpen Planning

The planning layer does not model pitcher substitution or bullpen management. In games where pitching decisions dominate — common in late innings — the recommendations from Module 5 are necessarily incomplete. Adding this capability would require a dedicated pitching module with fatigue state, bullpen role availability, and matchup transition logic feeding into the planner.

Limitation 2: Input Sensitivity

All five modules are rule-driven or optimization-driven, meaning they do not degrade gracefully when input data is sparse or inconsistent. A player with missing OBP data may score artificially low in Module 1, and that distortion propagates through assignment, batting order, and planning. Future versions should include validation diagnostics, confidence flags on low-data scores, and user-facing warnings when recommendation reliability is uncertain [1], [2], [3].

Limitation 3: Static Rule and Weight Calibration

Rule thresholds and objective weights are configured defaults. While this keeps the system interpretable, recommendation quality depends on how well those defaults generalize to a given team and opponent. A calibration workflow — either manual scenario testing or lightweight data-driven tuning — would meaningfully improve adaptability without sacrificing interpretability.

Limitation 4: No External Benchmarking

The system was evaluated on project-internal scenario data, not on historical game logs or published lineup datasets. Generalization claims therefore cannot be made with confidence. Retrospective evaluation against real game outcomes, compared against simple heuristic baselines, would be the clearest path to understanding where the system adds practical value.

8. Individual Contributions

Both partners aimed to make equal contributions to this project. Kali Javetski contributed to architecture framing and module-level specification, and specifically focused on the checkpoint reporting, and final paper composition. Sylvia Burroughs led core implementation across all five modules and key integration work, built and stabilized module interfaces and test harnesses for pipeline chaining, and drove code-level quality and system readiness. The estimated split between partners was 50% on Kali Javetski and 50% on Sylvia Burroughs.

9. Conclusions and Future Work

This project shows that baseball and softball lineup optimization is a natural fit for a multi-paradigm AI approach. First-order logic handles interpretable matchup reasoning; a knowledge base encodes position-specific defensive expertise; constraint satisfaction enforces hard eligibility rules while optimizing assignment quality; a genetic algorithm efficiently searches a combinatorial batting order space; and a planning module turns the resulting lineup into game-state-responsive recommendations.

The most important outcome is that all five stages connect into a working, testable pipeline. Each module has a defined role and explicit interfaces, and integration tests confirm that the full Modules 1–5 chain produces valid outputs from realistic inputs.

The highest-priority next steps are a pitcher and bullpen planning module, expanded scenario datasets for sensitivity analysis, confidence flags on low-quality inputs, and explanation output showing why a specific recommendation was selected and what alternatives were considered. Position-switch uncertainty — an underexplored factor in lineup modeling [13] — is also a candidate for more careful treatment. Longer term, lightweight data-driven calibration of rule thresholds and fitness weights, while preserving the interpretability that makes the system practically useful, would strengthen generalization beyond the current project dataset.

10. References

- [1] FanGraphs, "On-Base Percentage (OBP)," FanGraphs Library. [Online]. Available: <https://library.fangraphs.com/offense/obp/>. [Accessed: Apr. 27, 2026].
- [2] FanGraphs, "Slugging Percentage (SLG)," FanGraphs Library. [Online]. Available: <https://library.fangraphs.com/offense/slgl/>. [Accessed: Apr. 27, 2026].
- [3] Baseball-Reference, "Major League Baseball Standard Fielding," Baseball-Reference.com. [Online]. Available: <https://www.baseball-reference.com/leagues/majors/field.shtml>. [Accessed: Apr. 27, 2026].
- [4] MLB, "Caught Stealing Percentage (CS%)," MLB Glossary. [Online]. Available: <https://www.mlb.com/glossary/standard-stats/caught-stealing-percentage>. [Accessed: Apr. 27, 2026].
- [5] P. Slowinski, "Positional Adjustment," FanGraphs Sabermetrics Library. [Online]. Available: <https://library.fangraphs.com/misc/war/positional-adjustment/>. [Accessed: Apr. 27, 2026].
- [6] FanGraphs Community, "How the Positional Adjustments Have Changed Over Time (Part 1)," FanGraphs Community Blog. [Online]. Available: <https://community.fangraphs.com/how-the-positional-adjustments-have-changed-over-time-part-1/>. [Accessed: Apr. 27, 2026].
- [7] T. M. Tango, M. G. Lichtman, and A. E. Dolphin, The Book: Playing the Percentages in Baseball. Washington, DC, USA: Potomac Books, 2007.
- [8] K. Javetski and S. Burroughs, "Baseball/Softball Lineup Optimization System," project README.md, 2026.
- [9] "Module Rubric Report — Module 5: Adaptive Planning," checkpoints/checkpoint_5_module_report.md, 2026.

- [10] "Module Rubric Report — Project (Full AI System)," checkpoints/project_module_rubric_report.md, 2026.
- [11] T. M. Tango, "Fielding Position Adjustments," Inside the Book. [Online]. Available: https://insidethebook.com/ee/index.php/site/article/fielding_position_adjustments. [Accessed: Apr. 27, 2026].
- [12] J. M. Bradbury, The Sabermetric Revolution: Assessing the Growth of Analytics in Baseball. Philadelphia, PA, USA: Univ. of Pennsylvania Press, 2023.
- [13] Baseball Prospectus, "Baseball Therapy: Learning A New Position Is Free," BaseballProspectus.com. [Online]. Available: <https://www.baseballprospectus.com/news/article/30076/baseball-therapy-learning-a-new-position-is-free/>. [Accessed: Apr. 27, 2026].